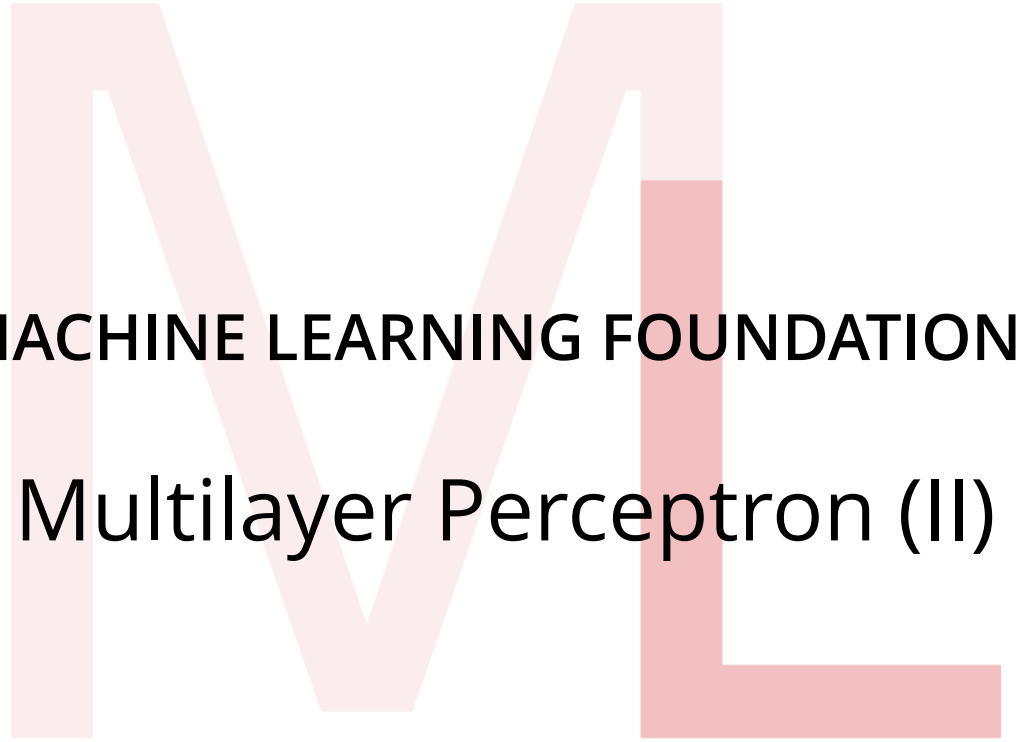
MACHINE LEARNING FOUNDATIONS

Multilayer Perceptron (II)



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Agenda

01 Recap

02 Deep neural networks

03 Vanishing/exploding gradients

04 Parameter initialization

Recap

- MLPs or Neural Networks with one hidden layer are able to approximate any function, due to the nonlinear activation function
  - However, the number of units and the training time can be incredible high
- Sometime we need to deal with more complex problems
  - The relation between the class output and the variable input is very nuclear
  - For example, can we express the probability of an image belonging to class “dog” as a function of the pixel intensities with only one nonlinear mapping?
- If the nonlinearity allows us to go from linear models to (somehow) complex problems…
  - Adding nonlinearities will allow us to deal with much complex problems

Neural Networks with arbitrary number of layers

- Once we have seen one-hidden-layers NNs, we can go deeper and add arbitrary number of layers
- For example, we can add a second hidden layer between the first hidden and the output layer
- Suppose a two-hidden-layer NN with h_1, h_2 units in the first and the second hidden layer, respectively
  - Parameters:
    - $W^{(1)} \in \mathbb{R}^{dxh_1}, b^{(1)} \in \mathbb{R}^{1xh_1}$
    - $W^{(2)} \in \mathbb{R}^{h_1xh_2}, b^{(2)} \in \mathbb{R}^{1xh_2}$
    - $W^{(3)} \in \mathbb{R}^{h_2xq}, b^{(3)} \in \mathbb{R}^{1xq}$

Neural Networks with arbitrary number of layers

- Calculations
  - Input: $X\left(H^{(0)}\right) \in \mathbb{R}^{nxd}$
  - $H^{(1)} = \phi(XW^{(1)} + b^{(1)}) \in \mathbb{R}^{nxh_1}$
  - $H^{(2)} = \phi(H^{(1)}W^{(2)} + b^{(2)}) \in \mathbb{R}^{nxh_2}$
  - $O = H^{(2)}W^{(3)} + b^{(3)} \in \mathbb{R}^{nxq}$

- Is interesting to see how the neural network transforms X into O
  - $O = H^{(2)}W^{(3)} + b^{(3)} = \phi\left(H^{(1)}W^{(2)} + b^{(2)}\right)\mathrm{W}^{(3)} + b^{(3)} =$
    $\phi\left(\phi\left(XW^{(1)} + b^{(1)}\right)W^{(2)} + b^{(2)}\right)W^{(3)} + b^{(3)}$
  - The transformation between X and O is “more” nonlinear than a one-hidden-layer

Neural Networks with arbitrary number of layers

- In general, we can think on an MLP with an arbitry number of layers (L) each one with $h_1, \dots, h_L$ number of units
- Parameters:
  - $W^{(i)} \in \mathbb{R}^{h_{i-1} x h_i}, b^{(i)} \in \mathbb{R}^{1 x h_i}, i \in \{1, \dots, L-1\}\, with\, h_0 = d$
  - $W^{(L)} \in \mathbb{R}^{h_{L-1} x q}, b^{(L)} \in \mathbb{R}^{1 x q}$
- Calculation:
  - $H^{(i)} = \phi(H^{(i-1)} W^{(i)} + b^{(i)}) \in \mathbb{R}^{n x h_i}, i \in \{1, \dots, L-1\} with\, H^0 = X$
  - $O = H^{L-1} W^{(L)} + b^{(L)} \in \mathbb{R}^{n x q}$

Neural Networks with arbitrary number of layers

- Once the forward propagation is completed, we start with the backpropagation
  - $\frac{\partial J}{\partial O} = \frac{\partial L}{\partial O} \in \mathbb{R}^{nxq}$
  - $\frac{\partial J}{\partial W^{(L)}} = \frac{\partial J}{\partial O}\frac{\partial O}{\partial W^{(L)}} = H^{(L-1)^\top}\frac{\partial J}{\partial O} \in \mathbb{R}^{h_{L-1}xq}$
  - $\frac{\partial J}{\partial H^{(L-1)}} = \frac{\partial J}{\partial O}\frac{\partial O}{\partial H^{(L-1)}} = \frac{\partial J}{\partial O}W^{(L-1)^\top} \in \mathbb{R}^{nxh_{L-1}}$
  - $\frac{\partial J}{\partial Z^{(L-1)}} = \frac{\partial J}{\partial H^{(L-1)}}\frac{\partial H^{(L-1)}}{\partial Z^{(L-1)}} = \frac{\partial J}{\partial H^{(L-1)}} \odot \phi\left(Z^{(L-1)}\right) \in \mathbb{R}^{nxh_{L-1}}$
  - $\frac{\partial J}{\partial W^{(L-1)}} = \frac{\partial J}{\partial Z^{(L-1)}}\frac{\partial Z^{(L-1)}}{\partial W^{(L-1)}} = H^{(L-2)^\top}\frac{\partial J}{\partial Z^{(L-1)}} \in \mathbb{R}^{h_{L-2}xh_{L-1}}$

Neural Networks with arbitrary number of layers

- In general, adding layers to a NN is easy
  - Each layer is responsible of its parameters $W^{(i)}, b^{(i)}$
  - In the forward pass the layer receives $H^{(i-1)}$, calculates $Z^{(i)}$ and sends $H^{(i)}$ to the next layer
  - In the backpropagation the layer receives $\frac{\partial J}{\partial H^{(i)}}$, calculates $\frac{\partial J}{\partial W^{(i)}}, \frac{\partial J}{\partial b^{(i)}}$ and sends $\frac{\partial J}{\partial H^{(i-1)}}$ to the next layer
- We now that adding more layers allows us to model much complex problems…
- So…let's add more and more layers!!

Neural Networks with arbitrary number of layers

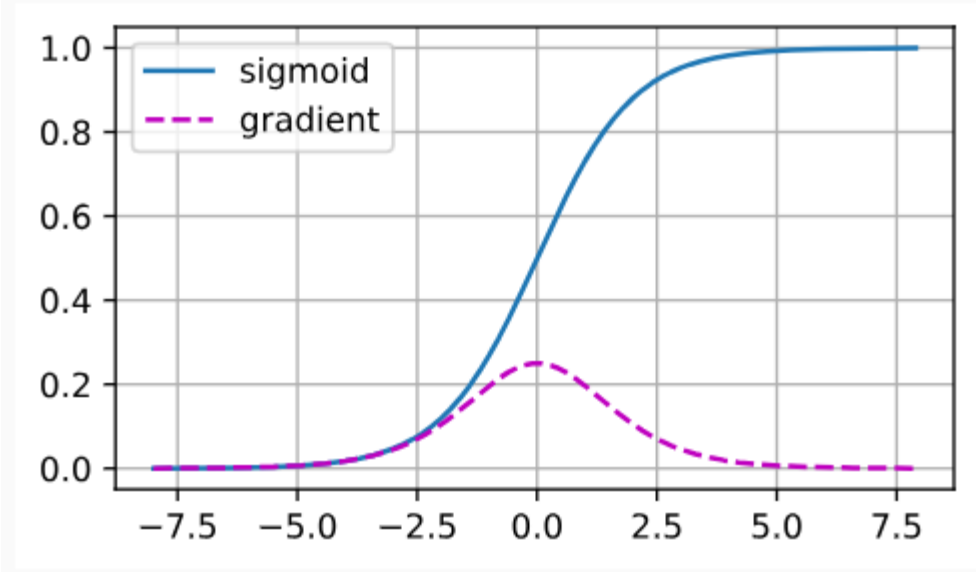
- Having very deep NNs is not free, since many problems arise when adding layers
- Main problems
  - Vanishing/exploding gradients
  - Complex (non convex) cost functions
  - Slow learning algorithms based on gradient descent
  - And, of course, overfitting
- And some others…
  - Large ammount of hyperparameters to tune
  - Need for bigger datasets

Vanishing/exploding gradients

- As we have seen, a L-hidden-layer neural network transforms input X into O by applying a concatenation of
  - Matrix multiplication
  - Activation function
- In the backpropagation, the derivative of the output with respect to $W^{(i)}$ is again calculate by multiplying many matrices starting from L to i
  - If matrices contain low numbers, the multiplication tends to zero
  - If matrices contain high numbers, the multiplication tends to grow

Vanishing/exploding gradients

- For example, when using sigmoid as activation function (now it is not common, but it was)
  - The derivative is zero almost everywhere
  - The gradents goes to zero at some point and we can not update any layer from that point



Vanishing/exploding gradients

- The opposite problem is also difficult to manage

```
M = torch.normal(0, 1, size=(4, 4))
print('a single matrix \n',M)
for i in range(100):
    M = M @ torch.normal(0, 1, size=(4, 4))
print('after multiplying 100 matrices\n', M)
```

```
a single matrix
 tensor([[-0.8755, -1.2171,  1.3316,  0.1357],
        [ 0.4399,  1.4073, -1.9131, -0.4608],
        [-2.1420,  0.3643, -0.5267,  1.0277],
        [-0.1734, -0.7549,  2.3024,  1.3085]])
after multiplying 100 matrices
 tensor([[-2.9185e+23,  1.3915e+25, -1.1865e+25,  1.4354e+24],
        [ 4.9142e+23, -2.3430e+25,  1.9979e+25, -2.4169e+24],
        [ 2.6578e+23, -1.2672e+25,  1.0805e+25, -1.3072e+24],
        [-5.2223e+23,  2.4899e+25, -2.1231e+25,  2.5684e+24]])
```

Parameter Initialization

- One way of addressing vanishing/exploding gradients is through careful initialization of parameters
- By default, we initialize the value of our weights using a normal distribution (mean 0 and variance σ^2)
- If the inputs have also mean 0 and variance γ^2 and are independent of weights and independent of each other, then

$$o_i = \sum_{j=1}^{n_{in}} w_{ij} x_j$$

  with again zero mean and variance $n_{in}\sigma^2\gamma^2$.

- One way to keep the variance of the forward propagation fixed is set $n_{in}\sigma^2 = 1$.

Parameter initialization

- Now, if we consider the backpropagation, we have the same issue with gradients
  - If we want to keep variance of gradients constant, we need to stablish $n_{out}\sigma^2 = 1$
- The problem arises when we want to satify both conditions simultaneously. Since this is not possible, one of the most well-known initialization is Xavier initialization, that tries to combine both equations and set

$$\frac{1}{2}(n_{in} + n_{out})^2\sigma^2 = 1 \quad or \quad \sigma = \sqrt{\frac{2}{n_{in} + n_{out}}}$$